

Vimu C SDK User Guide

Version 1.22

Vimu Electronic Technology

2026-1-20

www.vimu.top/

Update Log

V1.0 (2023.3.31)

- 1, New design

V1.1 (2023.5.22)

- 1, Add Logic Channel Trigger Source support

V1.2 (2023.6.26)

- 1, Add Watchdog Ctrl

V1.3 (2023.8.20)

- 1, Added MSO21 device support
- 2, Add DDS ARB and gating API

V1.4 (2023.11.06)

- 1, Linux system to increase stability
- 2, DLLTest demo supports re-insertion, automatic connection and acquisition

V1.5 (2023.12.14)

- 1, Add MSO10 and MSO21 V2 device support
- 2, Linux support read more than 4MB

V1.6 (2024.03.17)

- 1, DDS asdd burst APIs

V1.7 (2024.05.15)

- 1, Add trigger point read APIs

V1.8 (2024.07.15)

- 1, MSO41 support

V1.9 (2024.09.29)

- 1, MSO21 V3 和 MSO10 V2 support

V1.10 (2024.11.21)

- 1, Fixed not all channel acquisition and reading data misalignments
- 2, Fixed USB 3.0 linux bugs

V1.11 (2025.03.09)

- 1, Increased system stability

V1.20 (2025.07.28)

- 1, add dds and io api

V1.21 (2025.10.17)

- 1, add dds api
- 2, add more demos

V1.22 (2026.1.20)

- 1, add stream adc read api
- 2, Enhance trigger point reading API
- 3, Phase difference algorithm upgrade

Catalog

1.	Introduction	1
2.	SDK Directory Description	1
3.	Work style	3
3.1.	USB Connection Method	3
3.2.	MixCore Collection System	4
4.	System working block diagram	4
5.	Cyclic detection mode	5
6.	Callback function mode	7
7.	API	8
7.1.	Initialization and finish	8
7.2.	Equipment ID	9
7.3.	Equipment Reset	9
7.4.	Equipment Monitor	9
7.4.1.	callback function	9
7.4.2.	Event	10
7.4.3.	Loop Detect	10
7.4.4.	Scanning Devices	10
7.5.	Oscillograph	10
7.5.1.	Channel Num	10
7.5.2.	Storage Depth	11
7.5.3.	Capture Range	11
7.5.4.	Sample	11
7.5.5.	Trigger	12
7.5.6.	AC/DC	15
7.5.7.	Capture	16
7.5.8.	Capture Completion Notice	17
7.5.8.1.	callback function	17
7.5.8.2.	Event	17
7.5.8.3.	loop detect	18
7.5.9.	Data Read	18
7.6.	DDS	19
7.6.1.	Device	19
7.6.2.	Basic	20
7.6.3.	Sweep	22
7.6.4.	Burst	23
7.6.5.	Trigger and Output	25
7.6.6.	Synchronization	27
7.7.	IO	27
	Callback functions	27
	Event	27
	Loop Detect	28
7.8.	DAC	30
8.	Multiple devices	30

9.	Streaming mode is used	31
9.1.	Main function	31
9.2.	Device plug in callback function	31
9.3.	Data callback function.....	32
10.	Stream Mode API	33
10.1.	Working mode	33
10.2.	Collection range	33
10.3.	Sampling rate	34
10.4.	Capture	34
10.5.	Capture completion notification	34

1. Introduction

The Vimu C SDK serves as the standard DLL interface for the Vimu MSO mixed-signal oscilloscope/acquisition card and the Vimu MixCore acquisition system. Through this interface, direct hardware communication is possible, and when used with the open-source Vimulink platform, product design can be quickly completed.

This interface supports Windows systems (X86, X64, and ARM64) and Linux systems (X64, ARM32, and ARM64). For Linux systems, dynamic libraries are provided compiled with corresponding compilers for desktop Ubuntu, Kylin, small host Raspberry Pi, and industrial computers such as RK3568. If needed, you can contact us, and we provide technical support for compiling the corresponding dynamic libraries for specific GCC compilers.

2. SDK Directory Description

README.md	General instruction document	
SharedLibrary	vmsignal	Signal processing library header files
	VmMsoLib.h	Dynamic library header file
	VmMultMsoLib.h	Multi-device dynamic library header file
	Windows	Windows platform dynamic libraries
	RaspberryPi	Raspberry Pi platform dynamic libraries
	Ubuntu	Ubuntu platform dynamic libraries
	Linux	Embedded Arm64 platform dynamic libraries
vimu C SDK user guide.pdf	Detailed manual	
DebugView	Windows platform log collection software that can capture log information printed by DLLs	
C++	DllTest-Callback/ DllTestForMso41-Callback	C++ Console callback example
	DllTest-Polling	C++ Console loop detection example
	DLLMultDev-Stream	C++ Console stream mode example
	demo-VC	MFC example
	MultDevdemo-VC	MFC multi-device example
	MultDevSwitchdemo-VC	MFC real-time and stream mode switching example
	VimuLink by Qt	Open-source full-featured version example designed with Qt5 (includes oscilloscope, DDS, IO,

		FFT, and filter)
	How to run in the Raspberry.pdf	Examples of how to use C++ and Qt on Raspberry Pi
C#	NETTest-Callback	Examples of .Net console callback
	NETTest	Examples of .Net console loop detection
Labview	state_demo_for_two_channel	2-channel + 1 DDS demonstration example
	state_demo_for_four_channel	4-channel + 2 DDS demonstration example
	state_demo_for_four_channel_burst	DDS burst mode, example of triggering oscilloscope acquisition via IO
	How to run Labview demo.pdf	How to import dynamic libraries in LabVIEW
Python	test_callback.py/test_callback2.py	Python callback example
	test_polling.py	Python loop detection example
	test_multidev_stream_callback.py	Python streaming mode example

SharedLibrary Library selection:

Windows, Ubuntu, and Raspberry Pi platforms have dedicated directory libraries, and C examples can be opened and compiled directly. Qt examples require manually selecting the corresponding platform in the .pro file. As shown below.

```
linux {
    # 主机是 X64
    equals(QT_ARCH, x86_64) {
        message("Host is X64")

        CONFIG(debug, debug|release) {
            vmmoslib_dir = $$PWD/../Library/SharedLibrary/Ubuntu/X64/Debug/
            message("Linux X64 Debug: Using Debug libraries")
        } else {
            vmmoslib_dir = $$PWD/../Library/SharedLibrary/Ubuntu/X64/Release/
            message("Linux X64 Release: Using Release libraries")
        }
    }
    # 交叉编译 ARM64
    contains(QT_ARCH, arm64)|contains(QT_ARCH, aarch64) {
        message("Cross-compiling for ARM64")

        CONFIG(debug, debug|release) {
            # ARM64 Debug 库路径
            vmmoslib_dir = $$PWD/../Library/SharedLibrary/Ubuntu/aarch64-buildroot-linux-gnu-g++/Debug/
            message("ARM64 Debug: Using Debug libraries")
        } else {
            # ARM64 Release 库路径
            vmmoslib_dir = $$PWD/../Library/SharedLibrary/Ubuntu/aarch64-buildroot-linux-gnu-g++/Release/
            message("ARM64 Release: Using Release libraries")
        }
    }

    DEFINES += PLATFORM_ARM64
}
}
```

For other platforms, the Linux directory provides some compiler .so files, and you can choose to link with the .so files that match your compiler version or an older version. If none

of them work, you can contact us, and we will provide .so libraries for the corresponding compiler.s

VimuLink by Qt Open Platform:

VimuLink by Qt is an open-source Qt5 platform written in C++ and QML. This example can be compiled and run on Windows desktop platforms, Linux desktop platforms, and Linux embedded platforms.

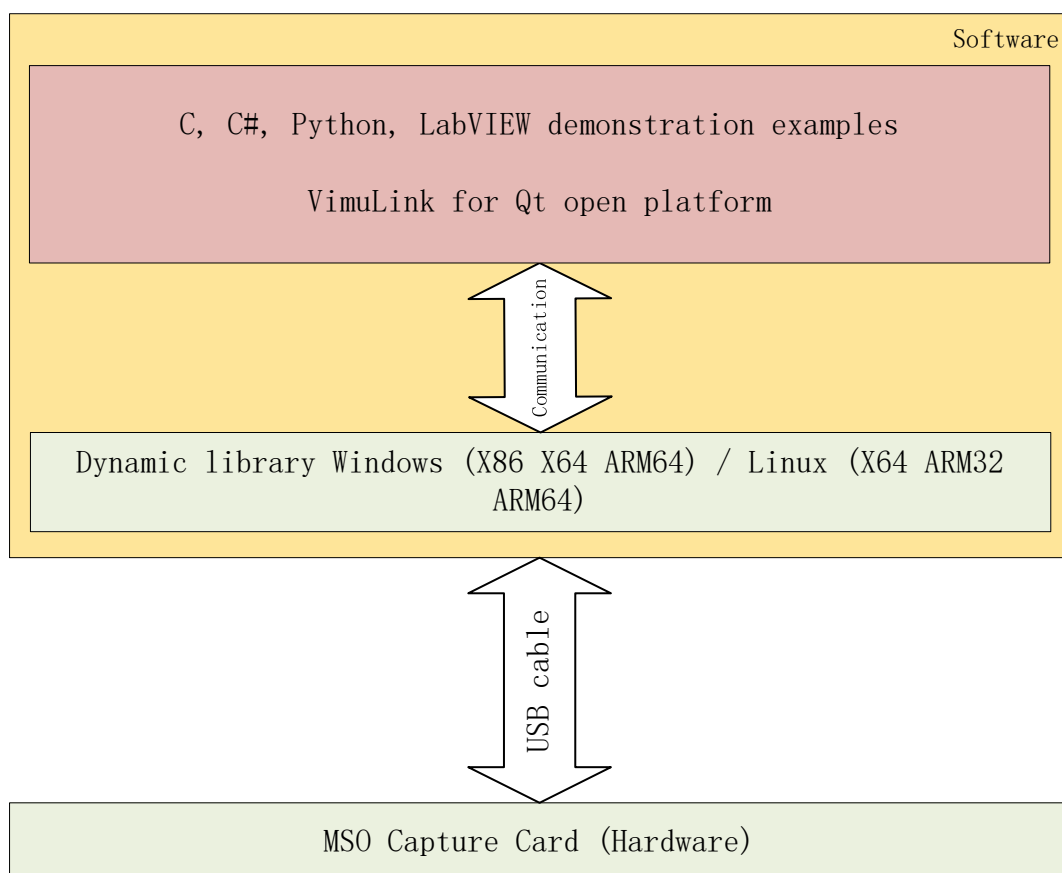
The example demonstrates how to use the oscilloscope in real-time mode and streaming mode to collect data, as well as real-time refresh plotting; it supports one-click switching between real-time mode and streaming mode. It showcases all the functions of DDS, including frequency sweep and burst mode operations. It also demonstrates IO input and output functionality, FFT calculation and plotting, reading of FDD files for filters, and real-time filter plotting and processing. It can greatly accelerate product development.

3. Work style

Users can directly download the SDK development package from the official website. The development package already includes the manual, dynamic library files for various platforms, and all the code for demonstration examples.

3.1. USB Connection Method

The MSO series oscilloscopes/data acquisition cards can connect to a computer, mini PC, or industrial PC via USB.



3.2. MixCore Collection System

The MixCore acquisition system is a collection system that integrates acquisition cards and industrial control computers.

Rich interfaces:

It provides interfaces for LCD screens, touchpads, HDMI, SD cards, USB, Ethernet, serial ports, etc.;

Multiple systems:

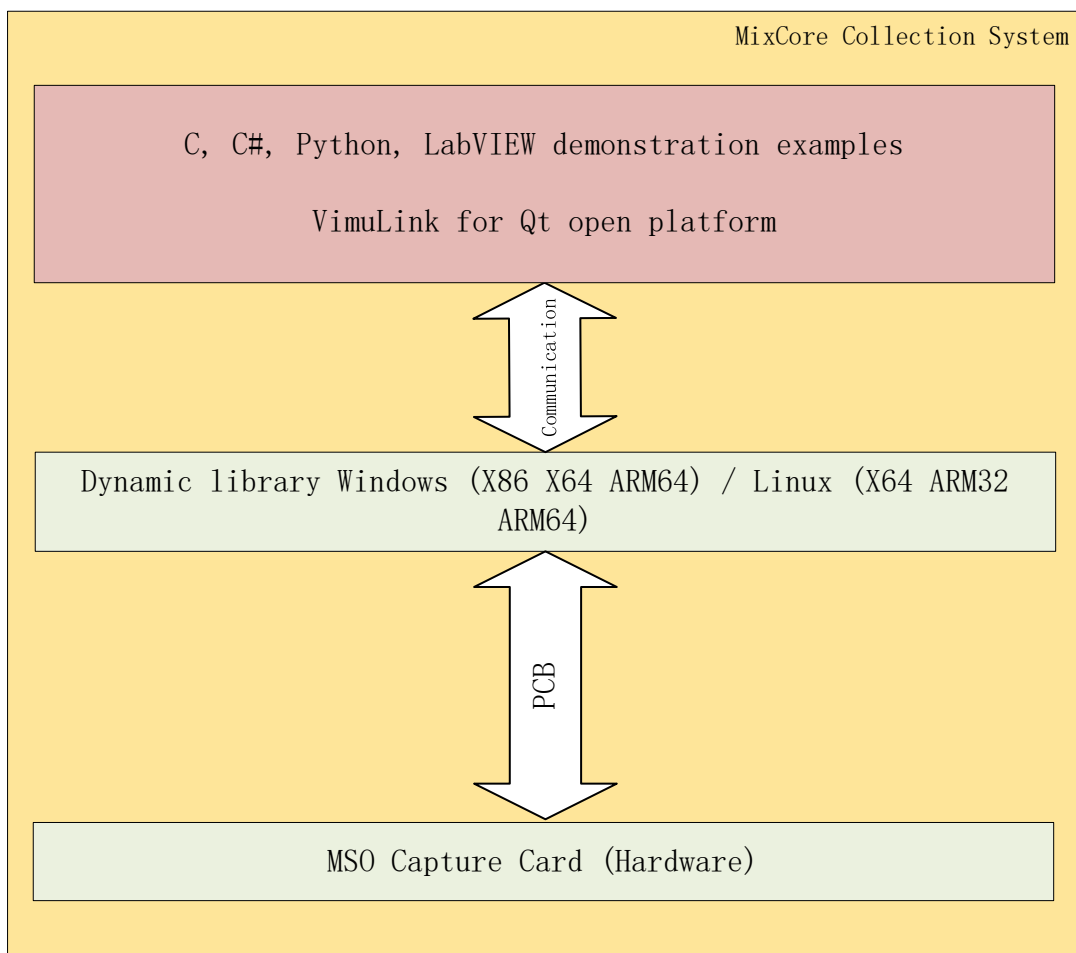
Supports Linux and Android systems. Combined with our provided Vimulink open-source platform, you can quickly complete verification and design of new products. (For Android system, please download VmMso Android SDK)

Powerful secondary development support:

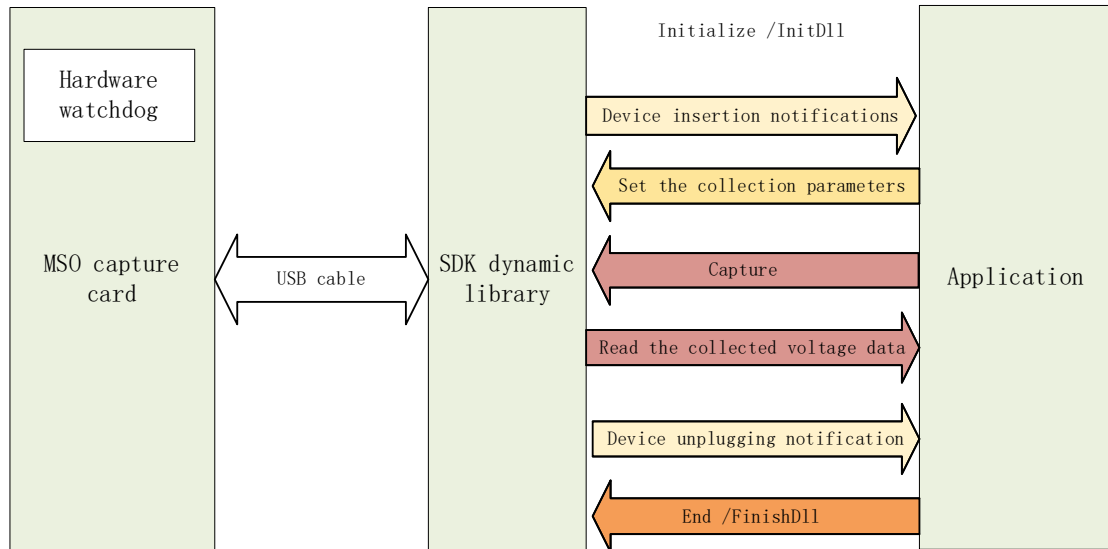
For the Linux system, we provide C dynamic libraries and demo examples; for the Android system, we provide AAR development packages and demo examples.

On the Linux system, Vimulink by Qt is a Qt5 open-source platform written in C and QML, allowing users to directly modify the platform to complete their own product designs.

On the Android system, Vimulink for Android is a native Java open-source platform, allowing users to directly modify the platform to complete their own product designs.



4. System working block diagram



MSO capture card: The hardware capture card part communicates with the host computer via USB

SDK dynamic library: Responsible for communicating with hardware; The parameters of the application are transmitted to the hardware card, and the acquisition data of the hardware card is returned to the application

Application: User-designed programs

Illustrate:

1. The parameter **watchdog enable** of InitDll can start the watchdog of the capture card, and remember to start the watchdog of the last delivered program. When debugging, the watchdog can be turned off as needed;
2. The SDK dynamic library thread is a separate thread. When using the callback mode, complex tasks cannot be handled in the callback function. If the callback function takes up too long and exceeds the threshold time of the watchdog, the hardware capture card will be restarted.

The working mode of the capture card can be roughly divided into "cycle detection" and "callback function" mode. When developing applications, you can choose according to actual needs, or you can mix them according to actual needs.

5. Cyclic detection mode

In the cyclic detection mode, the status of the capture card is queried through API polling, and then processed accordingly.

DllTest-Polling is an example of a circular detection. Here's the code:

```

int main()
{
    std::cout << "Vdso Test..." << std::endl;
    InitDll(1, 1);
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    ScanDevice();

    while (true)

```

```

{
    //device connection is successful?
    if(!devAvailable)
    {
        devAvailable = IsDevAvailable();

        //init functions
        if(devAvailable)
        {
            std::cout << "devAvailable start init" << std::endl;
            initFunction();
        }
        else
            std::this_thread::sleep_for(std::chrono::milliseconds(500));
    }

    if(devAvailable)
    {
        if (IsDataReady())
        {
            ReadDatas();
            NextCapture();
        }
        else if(IsIOReadStateReady())
        {
            std::cout << "io state " << std::hex << GetIOInState() << " \n";
        }
        else
        {
            //test device is active
            devAvailable = IsDevAvailable();
            if(!devAvailable)
                std::cout << "devAvailable is false" << std::endl;

            std::this_thread::sleep_for(std::chrono::milliseconds(200));
        }
    }
};
FinishDll();
std::cout << "...Vdso Test" << std::endl;
return 0;
}

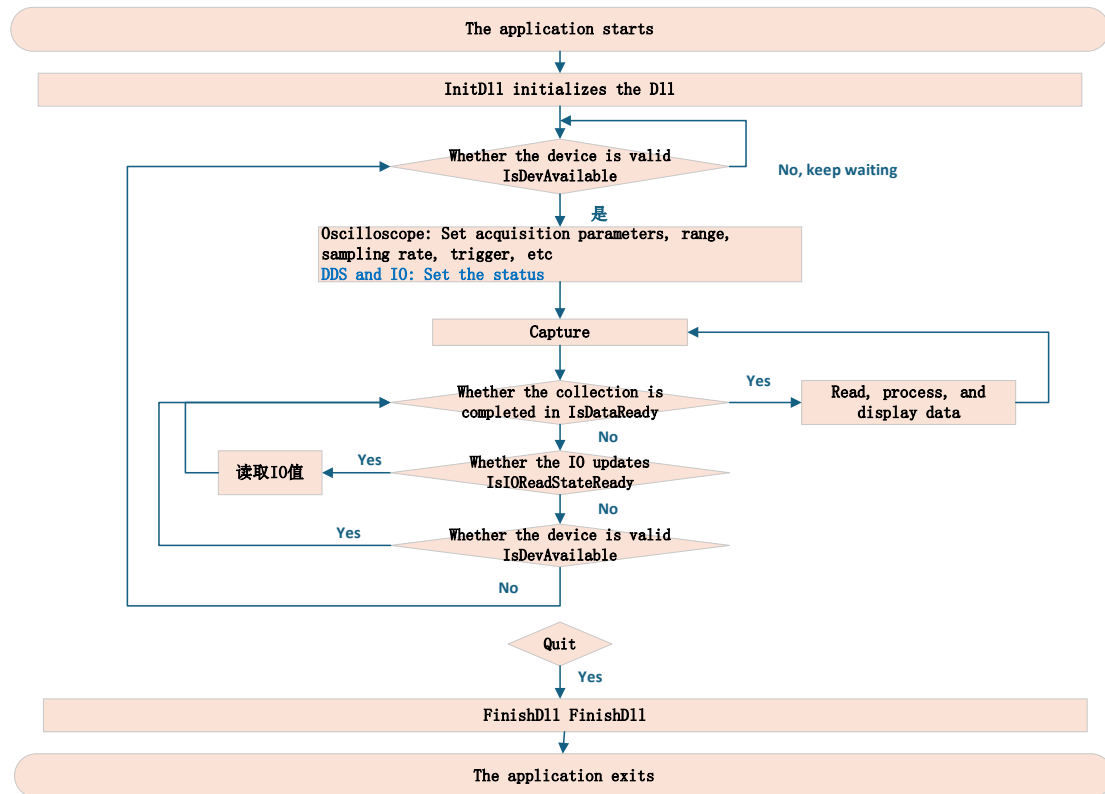
```

At first, check whether the device is plugged in, if not, the thread sleeps for 500ms, and then re-detects. When the device is detected, the device is initialized and Capture is invoked to

start collecting data.

Then, start polling to check whether the oscilloscope has completed the acquisition and whether the IO has been updated. If it is not updated, by the way, check whether the device is still plugged in.

The oscilloscope or IO has been updated, and the corresponding processing work will be done on the reading data, and then the next acquisition will be carried out. If a device is detected to be unplugged, it re-enters the device detection poll.



6. Callback function mode

The callback function mode is to register the callback function to obtain the status of the capture card, and then process it accordingly.

The callback function mode involves the synchronization of the capture card thread and the UI thread, and the synchronization between threads needs to be handled according to the actual situation when developing the program.

DllTest-Callback is an example of a callback function. Here's the code:

```

int main()
{
    std::cout << "Vdso Test..." << std::endl;

    InitDll(1, 1);

    //OSC
    SetDevNoticeCallBack(NULL, DevNoticeAddCallBack,
    DevNoticeRemoveCallBack);
  
```

```

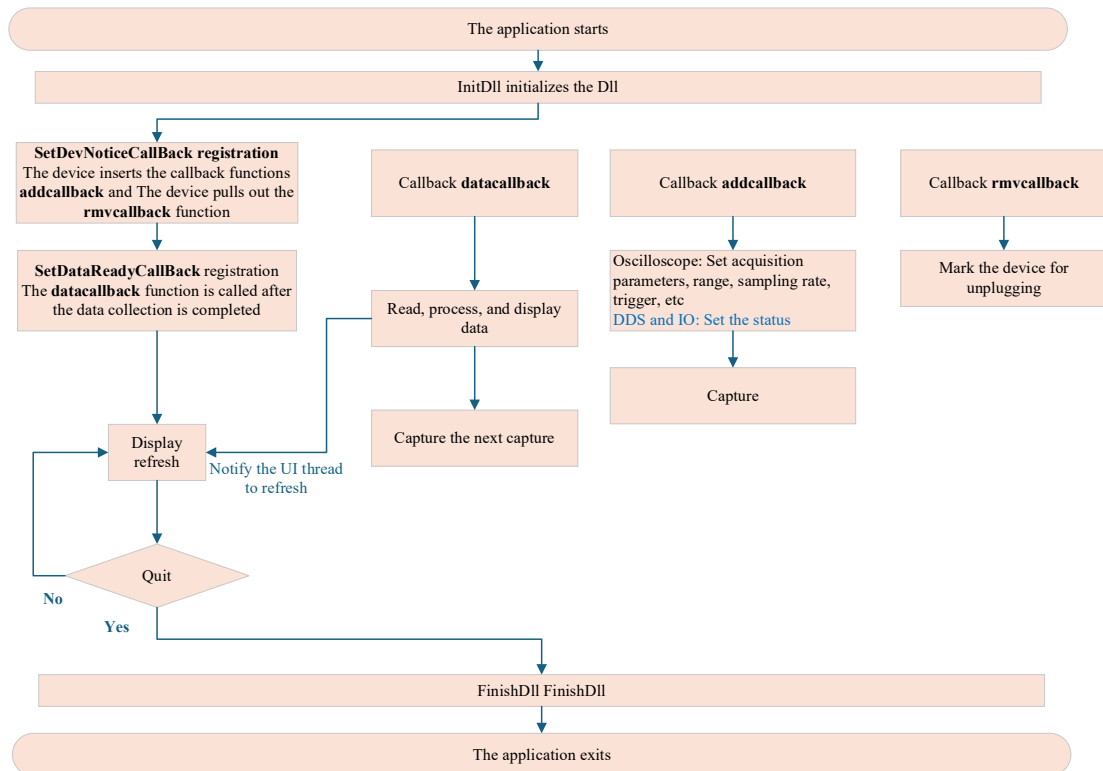
SetDataReadyCallBack(NULL, DevDataReadyCallBack);

std::this_thread::sleep_for(std::chrono::milliseconds(500));
ScanDevice();

while (true)
{
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
};

FinishDll();
std::cout << "...Vdso Test" << std::endl;
return 0;
}

```



The callback function **addcallback**: is responsible for initializing each function after the device is inserted, and then starts the collection.

The callback function **rmcallback** is responsible for marking and handling the status of the device after it is unplugged.

The callback function **datacallback** is used to read the collected data and then perform the next collection.

7. API

7.1. Initialization and finish

Call `InitDll ()` to complete the initialization of dynamic library, initialize memory and resources allocated for equipment monitoring and data reading.

int InitDll(unsigned int en_log , unsigned int en_hard_watchdog);

Description Dll initialization

Input: **log enable** 1 Enable Log
0 Not Enable Log
watchdog enable 1 Enable hard watchdog
0 Not Enable hard watchdog

Output: **Init Status**
Return value 1 Success
0 Failed

Call FinishDll () to complete the end of a dynamic library, freeing memory initialization and related resources in the application.

int FinishDll(void);

Description Dll finished

Input: -

Output: **-Finished Status**
Return value 1 Success
0 Failed

7.2. Equipment ID

The device ID is a 64-bit integer.

int GetOnlyId0(void);

Description This routines return device id(0-31)

Input: -

Output: - **Device ID(0-31)**

int GetOnlyId1(void);

Description This routines return device id(32-63)

Input: -

Output: - **Device ID(32-63)**

7.3. Equipment Reset

int ResetDevice(void);

Description This routines reset device

Input: -

Output: - **Return value** 1 success
0 failed

7.4. Equipment Monitor

when the device is detected, the dll have three ways to notify the main program, callback function, set Event and the main program loop detection.

7.4.1. callback function

When equipment is detect a function "**addcallback**" in the application can be called; when equipment is removed a function "**rmvcallback**" in the application can be called. The DLL has a function pointer which has to be set to these function, using

void SetDevNoticeCallBack(void* ppara, AddCallBack addcallback, RemoveCallBack rmvcallback);

Description This routines sets the callback function of equipment status changed.

Input: **ppara** the parameter of the callback function
 addcallback a pointer to a function with the following prototype:
 void AddCallBack(void * **ppara**)
 rmvcallback a pointer to a function with the following prototype:
 Void RemoveCallBack(void * **ppara**)

Output -

7.4.2. Event

When equipment is detect, an event "**addevent**" can be set by the DLL; when equipment is removed, an event "**rmvevent**" can be set by the DLL. The user must reset the event when the used them. The DLL has a function pointer which has to be set to these event using

void SetDevNoticeEvent(HANDLE addevent, HANDLE rmvevent);

Description This routines set the event handle, these will be set, when equipment status

changed.

Input: **addevent** the event handle
 rmvevent the event handle

Output -

7.4.3. Loop Detect

int IsDevAvailable();

Description This routines return the device is available or not.

Input: -

Output **Return value** 1 available
 0 not available

Note: Only need to use one of three ways. Callback and Event functions are asynchronous, more efficient; main program loop detection over a certain time needed to detect whether the device is inserted or removed.

7.4.4. Scanning Devices

The software is ready to start, and you can use ScanDevice to scan for USB devices that have been plugged in.

int ScanDevice ();

Description Rescan Device

Input: -

Output: -Rescan Status
 Return value 1 Successs
 0 Failed

7.5. Oscillograph

7.5.1. Channel Num

Get the number of device channels.

int GetOscChannelNum();

Description This routines get the num of osd channel.

Input:

Output Return

7.5.2. Storage Depth

Get the storage depth of the device in KB. For example, a return value of 2048 means 2048KB = 2MB.

unsigned int GetMemoryLength();

Description This routines get memory depth of equipment (KB).

Input: -

Output memory depth of equipment

7.5.3. Capture Range

Device with a programmable gain amplifier, when the signal acquisition time is less than the AD range, the signal amplification gain amplifier to use more AD digits, improving the quality of signal acquisition. DLL will adjusted the range of settings according to the pre-gain amplifier automatically.

int SetOscChannelRangemV (int channel, int minmv, int maxmv);

Description This routines set the range of input signal.

Input: **channel** the set channel

0 channel 1

1 channel 2

minmv the minimum voltage of the input signal (mV)

maxmv the maximum voltage of the input signal (mV)

Output **Return value** 1 Success

0 Failed

Note: The maximum range of the probe collection X1, the maximum voltage oscilloscope can capture. Like MSO20 is[-12000mV,12000mV].

Note: In order to achieve better waveform, you need to set the acquisition range, based on the magnitude of the measured waveform. When necessary, you can dynamically change the acquisition range.

7.5.4. Sample

int GetOscSupportSampleRateNum();

Description This routines get the number of samples that the equipment support.

Input: -

Output **Return value** the support sample number

int GetOscSupportSampleRates(unsigned int* sample, int maxnum);

Description This routines get support samples of equipment.

Input: **sample** the array store the support samples of the equipment

maxnum the length of the array

Output **Return value** the sample number of array stored

int SetOscSampleRate(unsigned int sample);

Description This routines set the sample.

Input: **sample** the set sample

Output **Return value** 0 Failed

other value new sample

unsigned int GetOscSampleRate();

Description This routines get the sample.

Input: -

Output **Return value** sample

7.5.5. Trigger

This feature requires hardware trigger support. The hardware trigger point is the intermediate data, such as the acquisition of 128K data, trigger point is the 64K point.

Trigger Mode

```
#define TRIGGER_MODE_AUTO 0
```

```
#define TRIGGER_MODE_LIANXU 1
```

Trigger Style

```
#define TRIGGER_STYLE_NONE 0x0000 //not trigger
```

```
#define TRIGGER_STYLE_RISE_EDGE 0x0001 //Rising edge
```

```
#define TRIGGER_STYLE_FALL_EDGE 0x0002 //Falling edge
```

```
#define TRIGGER_STYLE_EDGE 0x0004 //Edge
```

```
#define TRIGGER_STYLE_P_MORE 0x0008 //Positive Pulse width(>)
```

```
#define TRIGGER_STYLE_P_LESS 0x0010 //Positive Pulse width(>)
```

```
#define TRIGGER_STYLE_P 0x0020 //Positive Pulse width(<=)
```

```
#define TRIGGER_STYLE_N_MORE 0x0040 //Negative Pulse width(>)
```

```
#define TRIGGER_STYLE_N_LESS 0x0080 //Negative Pulse width(>)
```

```
#define TRIGGER_STYLE_N 0x0100 //Negative Pulse width(<=)
```

int IsSupportHardTrigger();

Description This routines get the equipment support hardware trigger or not .

Input: -

Output **Return value** 1 support hardware trigger
0 not support hardware trigger

unsigned int GetTriggerMode();

Description This routines get the trigger mode.

Input: -

Output **Return value** TRIGGER_MODE_AUTO
TRIGGER_MODE_LIANXU

void SetTriggerMode(unsigned int mode);

Description This routines set the trigger mode.

Input: **mode** TRIGGER_MODE_AUTO
TRIGGER_MODE_LIANXU

Output -

unsigned int GetTriggerStyle();

Description This routines get the trigger style.

Input: -

Output	Return value	TRIGGER_STYLE_NONE TRIGGER_STYLE_RISE_EDGE TRIGGER_STYLE_FALL_EDGE TRIGGER_STYLE_EDGE TRIGGER_STYLE_P_MORE TRIGGER_STYLE_P_LESS TRIGGER_STYLE_P TRIGGER_STYLE_N_MORE TRIGGER_STYLE_N_LESS TRIGGER_STYLE_N
--------	---------------------	--

void SetTriggerStyle(unsigned int style);

Description This routines set the trigger style.

Input:	style	TRIGGER_STYLE_NONE TRIGGER_STYLE_RISE_EDGE TRIGGER_STYLE_FALL_EDGE TRIGGER_STYLE_EDGE TRIGGER_STYLE_P_MORE TRIGGER_STYLE_P_LESS TRIGGER_STYLE_P TRIGGER_STYLE_N_MORE TRIGGER_STYLE_N_LESS TRIGGER_STYLE_N
--------	--------------	--

Output -

int GetTriggerPulseWidthNsMin();

Description This routines get the min time of pulse width.

Input: -

Output Return min time value of pulse width(ns)

int GetTriggerPulseWidthNsMax();

Description This routines get the max time of pulse width.

Input: -

Output Return max time value of pulse width(ns)

int GetTriggerPulseWidthDownNs();

Description This routines get the down time of pulse width.

Input: -

Output Return down time value of pulse width(ns)

int GetTriggerPulseWidthUpNs();

Description This routines set the down time of pulse width.

Input: down time value of pulse width(ns)

Output -

void SetTriggerPulseWidthNs(int down_ns, int up_ns);

Description This routines set the up time of pulse width.

Input: up time value of pulse width(ns)

Output _

unsigned int GetTriggerSource();

Description This routines get the trigger source.

Input: -

Output **Return value** TRIGGER_SOURCE_CH1 0 //CH1
TRIGGER_SOURCE_CH2 1 //CH2
TRIGGER_SOURCE_LOGIC0 16 //Logic 0
TRIGGER_SOURCE_LOGIC1 17 //Logic 1
TRIGGER_SOURCE_LOGIC2 18 //Logic 2
TRIGGER_SOURCE_LOGIC3 19 //Logic 3
TRIGGER_SOURCE_LOGIC4 20 //Logic 4
TRIGGER_SOURCE_LOGIC5 21 //Logic 5
TRIGGER_SOURCE_LOGIC6 22 //Logic 6
TRIGGER_SOURCE_LOGIC7 23 //Logic 7

void SetTriggerSource(unsigned int source);

Description This routines set the trigger source.

Input: **source** TRIGGER_SOURCE_CH1 0 //CH1
TRIGGER_SOURCE_CH2 1 //CH2
TRIGGER_SOURCE_LOGIC0 16 //Logic 0
TRIGGER_SOURCE_LOGIC1 17 //Logic 1
TRIGGER_SOURCE_LOGIC2 18 //Logic 2
TRIGGER_SOURCE_LOGIC3 19 //Logic 3
TRIGGER_SOURCE_LOGIC4 20 //Logic 4
TRIGGER_SOURCE_LOGIC5 21 //Logic 5
TRIGGER_SOURCE_LOGIC6 22 //Logic 6
TRIGGER_SOURCE_LOGIC7 23 //Logic 7

Output -

Note: If the logic analyzer and IO are multiplexed (for example, MSO20, MSO21), the corresponding IO needs to be turned on and set to the input state.

int GetTriggerLevelmV();

Description This routines get the trigger level.

Input: -

Output **Return value** level (mV)

void SetTriggerLevelmV (int level, int sense);

Description This routines set the trigger level and sense.

Input: level (mV)

sense (mV)

Output -

int GetTriggerSensemV();

Description This routines get the trigger sense.

Input: -

Output **Return value** Sense (mV)

Note: Trigger sensitivity can make the trigger stability better, for example, setting the trigger to the rising edge, the trigger level to 1V, and the trigger sensitivity to 100mV. After the acquisition card detects 1V, it will continue to detect 1.1V, and if it is reached, it will be considered to be triggered to meet the conditions; if it does not detect 1.1V, it will be considered that it is not satisfied, which should be glitch interference.

bool IsSupportPreTriggerPercent();

Description This routines get the equipment support Pre-trigger Percent or not .

Input: -

Output Return value 1 support
0 not support

int GetPreTriggerPercent();

Description This routines get the Pre-trigger Percent.

Input: -

Output Return value Percent (5-95)

void SetPreTriggerPercent(int front);

Description This routines set the Pre-trigger Percent.

Input: Percent (5-95)

Output -

int IsSupportTriggerForce();

Description This routines get the equipment support trigger force or not.

Input: -

Return value 1 support
0 not support

void TriggerForce();

Description This routines force capture once.

Input: -

Output: -

7.5.6. AC/DC

int IsSupportAcDc(unsigned int channel);

Description This routines get the device support AC/DC switch or not.

Input: channel 0 :channel 1
1 :channel 2

Output **Return value** 0 : not support AC/DC switch

1 : support AC/DC switch

void SetAcDc(unsigned int channel, int ac);

Description This routines set the device AC coupling.

Input: channel 0 :channel 1

1 :channel 2

ac 1 : set AC coupling

0 : set DC coupling

Output -

int GetAcDc(unsigned int channel,);

Description This routines get the device AC coupling.

Input: channel 0 :channel 1

1 :channel 2

Output **Return value** 1 : AC coupling

0 : DC coupling

7.5.7. Capture

Call capture function to begin collecting data, **length** is the length you want to capture, using K Units, such as length = 16, is 16K 16384points. For sample rate greater than or equal the length of the depth of the collection is stored, take the minimum **length** and depth of storage;For the sampling rate is less than the memory depth, take the minimum **length** and one second data collection length. **force_length** can be forced to cancel the limit of only 1 seconds to be collected.

int Capture(int length, char capture_channel,char force_length);

Description This routines set the capture length and start capture.

Input: **length** capture length(KB)

capture_channel

ch1=0x0001 ch2=0x0002 ch3=0x0004 ch4=0x0008 logic=0x0100

ch1+ch2 0x0003

ch1+ch2+ch3 0x0007

ch1+logic 0x1001

force_length 1: force using the length, no longer limits the max collection

1 seconds

Output **Return value** the real capture length(KB)

When using normal trigger mode (TRIGGER_MODE_LIANXU). The collection command was sent, and the data notification that the collection was complete has not been received. Now, you want to stop the software.

1. Recommended method: You change the trigger mode to TRIGGER_MODE_AUTO, wait for the data notification to be collected, and then stop the software.

2. Use **AbortCapture**.

DLL_API int WINAPI AbortCapture();

Description This routines set the abort capture

Output Return value 1:success 0:failed

Description	This routines get memory depth of equipment (KB).
-------------	---

Output	memory depth of equipment(KB)
--------	-------------------------------

int IsSupportRollMode();

Input: -

Output	Return value 1 support roll mode 0 not support roll mode
--------	--

Description This routines enable or disenable the equipment into roll mode.

Input: -

Output **Return value** 1 success
 0 failed

when capture is complete, the dll have three ways to notify the main program, callback function, set Event and the main program loop detection.

when capture is complete, if the callback function **"datacallback"** is registered, it will be called. The DLL has a function pointer which has to be set to this function, using

void SetDataReadyCallBack(void* ppara, DataReadyCallBack datacallback);

Description	This routines sets the callback function of capture complete.
-------------	---

Input:	ppara	the parameter of the callback function
	datacallback	a pointer to a function with the following prototype:
		void DataReadyCallBack (void * ppara)

Output -

when capture is complete, if the Event handle "**dataevent**" is registered, it will be set. The user must reset the event when the used it. he DLL has a function pointer which has to be set to this function, using

void SetDevDataReadyEvent(HANDLE dataevent);

Description	This routines set the event handle, these will be set, when capture complete
-------------	--

Input: **dataevent** the event handle

Output -

7.5.8.3. loop detect

int IsDataReady();

Description This routines return the capture is complete or not.

Input: -

Output **Return value** 1 complete
0 not complete

Note: Only need to use one of three ways. Callback and Event functions are asynchronous, more efficient; main program loop detection over a certain time needed to detect whether the capture is complete or not.

7.5.9. Data Read

unsigned int ReadVoltageDatas(char channel, double* buffer,unsigned int length);

Description This routines read the voltage datas. (V)

Input: **channel** **read channel** 0 :channel 1
1 :channel 2

buffer the buffer to store voltage datas

length the buffer length

Output **Return value** the read length

unsigned int ReadVoltageDatasTriggerPoint();

Description This routines read the trigger location where the data collected

Input:

Output **Return value** the trigger points

int IsVoltageDatasOutOfRange(char channel);

Description This routines return the voltage datas is out range or not.

Input: **channel** **read channel** 0 :channel 1
1 :channel 2

Output **Return value** 0 :not out range
1 :out range

double GetVoltageResolution(char channel);

Description This routines return the current voltage resolution value

One ADC resolution for the voltage value:

Full scale is 1000mv

the ADC is 8 bits

voltage resolution value = 1000mV/256

Input: **channel** **read channel** 0:channel 1
1:channel 2

Output **Return value** voltage resolution value

unsigned int ReadLogicDatas(unsigned char* buffer, unsigned int length);

Description This routines read the logic data of mso.

Input:

buffer the buffer to store logic datas

length the buffer length

Output Return value the read length

unsigned int ReadLogicDatasTriggerPoint();

Description This routines read the trigger location where the data collected

Input:

Output **Return value** the trigger points

7.6.DDS

7.6.1. Device

int IsSupportDDSDevice();

Description This routines get support dds or not

Input: -

Output **Return value** support dds or not

int GetDDSDepth();

Description This routines set dds depth

Input:

Output: **Return value** depth

void DDSOutputEnable(unsigned char channel_index , int enable);

Description This routines enable dds output or not

Input: **channel_index** 0 : channel 1
 1 : channel 2

enable 1 enable
 0 not enable

Output: -

int IsDDSOutputEnable(unsigned char channel_index);

Description This routines get dds output enable or not

Input: **channel_index** 0 : channel 1
 1 : channel 2

Output **Return value** dds enable or not

void SetDDSOutMode(unsigned char channel_index, unsigned int out_mode);

Description This routines set dds out mode

Input: **channel_index** 0 :channel 1
 1 :channel 2
 out_mode DDS_OUT_MODE_CONTINUOUS 0x00
 DDS_OUT_MODE_SWEEP 0x01
 DDS_OUT_MODE_BURST 0x02

Output

unsigned int GetDDSOutMode(unsigned char channel_index);

Description This routines get dds out mode

arb_buffer the dac buffer
arb_buffer_length the dac buffer length need equal to the dds depth

Output: -

void SetDDSFreq(unsigned char channel_index, double freq);

Description This routines set frequency

Input: **channel_index** 0 :channel 1
 1 :channel 2

Input: **freq** frequency

Output: -

double GetDDSFreq(unsigned char channel_index);

Description This routines get frequency

Input: **channel_index** 0 :channel 1
 1 :channel 2

Output: **freq** frequency

void SetDDSDutyCycle(unsigned char channel_index, double cycle);

Description This routines set duty cycle

Input: **channel_index** 0 :channel 1
 1 :channel 2

Input: **cycle** duty cycle

Output: -

double GetDDSDutyCycle(unsigned char channel_index);

Description This routines get duty cycle

Input: **channel_index** 0 :channel 1
 1 :channel 2

Output: **cycle** duty cycle

void SetDDSPHase(unsigned char channel_index, double phase);

Description This routines set phase

Input: **channel_index** 0 :channel 1
 1 :channel 2

phase phase

Output: -

double GetDDSPHase(unsigned char channel_index);

Description This routines get phase

Input: **channel_index** 0 :channel 1
 1 :channel 2

Output: **phase** phase

int GetDDSCurBoxingAmplitudeMv(unsigned int boxing);

Description This routines get dds amplitdude of wave

Input: **boxing** BX_SINE~BX_ARB

Output: Return the amplitdude(mV) of wave

void SetDDSAmplitudeMv(unsigned char channel_index, int amplitdude);

Description This routines set dds amplitdude(mV)

Input: **channel_index** 0 :channel 1

1 :channel 2

amplitdude amplitdude(mV)

Output: -

int GetDDSAmplitudeMv(unsigned char channel_index);

Description This routines get dds amplitdude(mV)

Input: **channel_index** 0 :channel 1

1 :channel 2

Output: return amplitdude(mV)

int GetDDSCurBoxingBiasMvMin(unsigned int boxing);

int GetDDSCurBoxingBiasMvMax(unsigned int boxing);

Description This routines get dds bias of wave

Input: **boxing** BX_SINE~BX_ARB

Output: Return the bias(mV) range of wave

void SetDDSBiasMv(unsigned char channel_index, int bias);

Description This routines set dds bias(mV)

Input: **channel_index** 0 :channel 1

1 :channel 2

bias bias(mV)

Output: -

int GetDDSBiasMv(unsigned char channel_index);

Description This routines get dds bias(mV)

Input: **channel_index** 0 :channel 1

1 :channel 2

Output: Return the bias(mV) of wave

7.6.3. Sweep

void SetDDSSweepStartFreq(unsigned char channel_index, double freq);

Description This routines set dds sweep start freq

Input: **channel_index** 0 :channel 1

1 :channel 2

freq

Output: -

double GetDDSSweepStartFreq(unsigned char channel_index);

Description This routines get dds sweep start freq

Input: **channel_index** 0 :channel 1
 1 :channel 2

Output: **freq**

void SetDDSSweepStopFreq(unsigned char channel_index, double freq);

Description This routines set dds sweep stop freq

Input: **channel_index** 0 :channel 1
 1 :channel 2

freq

Output: -

double GetDDSSweepStopFreq(unsigned char channel_index);

Description This routines get dds sweep stop freq

Input: **channel_index** 0 :channel 1
 1 :channel 2

Output: **freq**

void SetDDSSweepTime(unsigned char channel_index, unsigned long long int time_ns);

Description This routines set dds sweep time

Input: **channel_index** 0 :channel 1
 1 :channel 2

time/ns

Output: -

unsigned long long int GetDDSSweepTime(unsigned char channel_index);

Description This routines get dds sweep time

Input: **channel_index** 0 :channel 1
 1 :channel 2

Output: **time/ns**

void SetDDSBurstStyle(unsigned char channel_index, int style);

Description This routines set dds burst style

Input: **channel_index** 0 :channel 1
 1 :channel 2

style 0--n loops
 1--gate

Output: -

7.6.4. Burst

int GetDDSBurstStyle(unsigned char channel_index);

Description This routines get dds burst style

<http://www.vimu.top/>

Output: -

unsigned long long int GetDDSBurstPeriodNs(unsigned char channel_index);

Description This routines get dds burst period(ns)

Input: **channel_index** 0 :channel 1
1 :channel 2

Output: **ns**

void SetDDSBurstDelayNs(unsigned char channel_index, unsigned long long int ns);

Description This routines set dds burst delay time(ns)

Input: **channel_index** 0 :channel 1
1 :channel 2

ns

Output: -

unsigned long long int GetDDSBurstDelayNs(unsigned char channel_index);

Description This routines get dds burst delay time(ns)

Input: **channel_index** 0 :channel 1
1 :channel 2

Output: **ns**

7.6.5. Trigger and Output

void SetDDSTriggerSource(unsigned char channel_index, unsigned int src);

Description This routines set dds trigger source

Input: **channel_index** 0 : channel 1
1 : channel 2
src 0 : internal
1 : external
2 : manual

Output: -

unsigned int GetDDSTriggerSource(unsigned char channel_index);

Description This routines get dds trigger source

Input: **channel_index** 0 : channel 1
1 : channel 2

Output: **trigger source** 0 : internal
1 : external
2 : manual

void SetDDSTriggerSourceIo(unsigned char channel_index, uint32_t io);

Description This routines set dds trigger source io

Input: **channel_index** 0 : channel 1
1 : channel 2

io 0 : DIO0

 7 : DIO7

Output: -

Note: You need to use the DIO API to set the corresponding DIO to the input/output state

uint32_t GetDDSTriggerSourceIo(unsigned char channel_index);

Description This routines get dds trigger source io

Input: **channel_index** 0 : channel 1
 1 : channel 2
 Output: **trigger source io** 0 : DIO0

 7 : DIO7

void SetDDSTriggerSourceEnge(unsigned char channel_index, unsigned int enge);

Description This routines set dds trigger source enge

Input: **channel_index** 0 : channel 1
 1 : channel 2
 enge 0 : rising
 1 : falling

Output: -

unsigned int GetDDSTriggerSourceEnge(unsigned char channel_index);

Description This routines get dds trigger enge

Input: **channel_index** 0 : channel 1
 1 : channel 2
 Output: **enge** 0 : rising
 1 : falling

void SetDDSOutputGateEnge(unsigned char channel_index, unsigned int enge);

Description This routines set dds output gate enge

Input: **channel_index** 0 : channel 1
 1 : channel 2
 enge 0 : close
 1 : rising
 2 : falling

Output: -

unsigned int GetDDSOutputGateEnge(unsigned char channel_index);

Description This routines get dds output gate enge

Input: **channel_index** 0 : channel 1
 1 : channel 2
 Output: **enge** 0 : close
 1 : rising

2 : falling

void DDSManualTrigger(unsigned char channel_index);

Description This routines manual trigger dds

Input: **channel_index** 0 : channel 1
 1 : channel 2

Output: -

7.6.6. Synchronization

void WINAPI DDSMultSyn();

Description This routines synchronized multiple devices

Input:

Output: -

7.7.IO

int IsSupportIODevice();

Description This routines get support IO ctrl or not

Input: -

Output Return value support io ctrl or not

int GetSupportIoNumber();

Description This routines get support io nums of equipment.

Output Return value the sample number of io nums

When IO is set as input, there are three ways to read the IO status, drop the function, trigger the Event, and the main program loop detection.

Callback functions

The SDK will read the IO status periodically, and if the main program registers the fallback function "datacallback", it will be called. DLLs have a function dedicated to setting this fallback function.

void SetIOReadStateCallBack(void* ppara, IOReadStateCallBack callback);

Description This routines sets the callback function of read io status.

Input: **ppara** the parameter of the callback function

callback a pointer to a function with the following prototype

Event

The SDK will read the IO status regularly, and if the main program registers the Event handle "dataevent", it will be set. It should be noted that after the main program detects the Event, it needs to reset the Event. The DLL has a function specifically for setting this Event handle

void SetIOReadStateReadyEvent(HANDLE dataevent);

Description This routines set the event handle, these will be set, when capture complete

Input: **dataevent** the event handle

Output -

Loop Detect

int IsIOReadStateReady();

Description This routines return read io is complete or not.

Input: -

Output **Return value** 1 complete
0 not complete

Note: Only need to use one of three ways. Callback and Event functions are asynchronous, more efficient; main program loop detection over a certain time needed to detect whether the capture is complete or not.

void IOEnable(unsigned char channel, unsigned char enable);

Description This routines set io enable or not

Input: **channel** dio0 0
dio1 1
dio2 2
.....
enable not enable 0
enable 1

Output: -

unsigned char IsIOEnable(unsigned char channel);

Description This routines get io enable or not

Input: **channel** dio0 0
dio1 1
dio2 2
.....

Output: **return** not enable 0 or enable 1

void SetIOInOut(unsigned char channel, unsigned char inout);

Description This routines set io in or out

Input: **channel** dio0 0
dio1 1
dio2 2
.....

inout in 0
out 1

Output: -

unsigned char GetIOInOut(unsigned char channel);

Description This routines get io in or out

Input: **channel** dio0 0
dio1 1


```

        dio2 2
        .....
Output:    return  in 0
           out 1

```

void SetIOOutState(unsigned char channel, unsigned char state);

Description This routines set io state

```

Input:      channel dio0 0
            dio1 1
            dio2 2
            .....
State 0--0
        1--1
        2--z
        3--pulse
        4--dds gate

```

Output: -

void SendAutoReadIOInTimeMs(uint32_t ms);

Description This routines set auto read thread io in state time interval

Input: ms the time min is 100ms

IO input state reading, the default read timing is 500ms, and the reading timing can be modified through this function.

unsigned int SendReadIOInStateCmd();

Description This routines send read hardware io cmd

Note:When sent and the hardware has returned, IsIOReadStateReady returns true

If 100ms cannot meet the real-time reading IO requirements, you can use this function to manually send the read hardware IO input state command, and after the reading is completed, IsIOReadStateReady becomes the true state. Use GetIOInState to read the current IO input state.

unsigned int GetIOInState();

Description This routines get io state

If the SetIOReadStateCallBack setting callback function is used, IOReadStateCallBack will directly notify the IO input status; If use SetIOReadStateReadyEvent and IsIOReadStateReady to read the query, you need to call GetIOState to get the IO input status

Output: return io state

void SetIOPulseData(unsigned char channel, double freq, double duty);

Description This routines set io pulse freq and duty

```

Input:      freq  freq(hz)
           duty 5~95

```

double GetIOPulseFreq(unsigned char channel);

Description This routines get io pulse freq

Output: freq

double GetIOPulseDuty(unsigned char channel);

Description This routines get io pulse duty

Output: duty 5~95

void SetIOPulseSyn();

Description This routines set io pulse syn output

Input:

Output:

7.8. DAC

Some devices have a DAC interface and can output an adjustable voltage signal of 0 to 3V.

void DACEnable(unsigned char channel, unsigned char enable);

Description This routines set dac enable or not

Input: channel channel number

enable not enable 0

enable 1

Output: -

unsigned char IsDACEnable(unsigned char channel);

Description This routines get dac enable or not

Input: channel channel number

Output: not enable 0

enable 1

void SetDACmV(unsigned char channel, int vol_mv);

Description This routines set dac vol(mv)

Input: channel channel number

Enable not enable 0

enable 1

Output: -

int GetDACmV(unsigned char channel_index);

Description This routines get dac vol(mv)

Input: channel channel number

Output: vol/mv

8. Multiple devices

Vimu Mult C SDK is a standard dynamic library based on the MSO C SDK that adds multi-device support APIs. The functional APIs for device control, oscilloscope, DDS, and IO have the same names and usage as in the vimu C SDK, with the only difference being the

addition of a device ID parameter, `unsigned int dev\_id`.

Compared to the vimu C SDK, vimu Mult C SDK adds stream mode functionality.

9. Streaming mode is used

Streaming modes require faster data processing to prevent data from accumulating into memory, and all streaming modes only support callback function mode.

9.1. Main function

```
int main()
{
    //init dll
    InitDll(1, 1);
    //Set up device and data callback functions
    SetDevNoticeCallBack(NULL, mDevNoticeAddCallBack,
mDevNoticeRemoveCallBack);
    SetStreamDataReadyCallBack(NULL, mStreamDataReadyCallBack);
    //Scan for devices that have been plugged in
    ScanDevice();
    //Main cycle
    while (true)
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        //Exit when the collection is complete
        if(capture_ok)
            break;
    };
    //Free up resources
    FinishDll();
    return 0;
}
```

9.2. Device plug in callback function

```
void CALLBACK mDevNoticeAddCallBack(void* ppara, unsigned int dev_id)
{
    //DDS init
    DDSInit(dev_id, 0, DDS_OUT_MODE_CONTINUOUS);

    //IO init
    IOInit(dev_id);

    //Select the working mode
    SetOscCaptureMode(dev_id, 1);
    //set input range
    SetStreamChannelRangemV(dev_id, 0, -10000, 10000);
    SetStreamChannelRangemV(dev_id, 1, -10000, 10000);
}
```

```

//sample rate
int sample_num = GetStreamSupportSampleRateNum(dev_id);
if (sample != NULL)
{
    delete[]sample;
    sample = NULL;
}
sample = new unsigned int[sample_num];
//read support sample rate
if (GetStreamSupportSampleRates(dev_id, sample, sample_num))
{
    for (int i = 0; i < sample_num; i++)
        std::cout << std::dec << sample[i] << '\n';
    std::cout << std::endl;
}
// Record the device and initialize the acquisition status
m_dev_id = dev_id;
capture_ok = false;
capture_ok_mask = 0;

// Using a 1M sampling rate, 10M data is collected
uint64_t length = 1024*1024*10; //10M
StreamCapture(m_dev_id, length/1024, capture_channel_mask, 1000000);
}

```

9.3. Data callback function

```

void CALLBACK mStreamDataReadyCallBack(void* ppara, unsigned int dev_id,
unsigned char channel_index, double* buffer, unsigned int buffer_length, unsigned int
failed, unsigned int success, unsigned long long int need_total_sample, unsigned long
long int total_sample, unsigned long long int memoryuse)
{

```

//Note: The callback function should not handle complex tasks, and if it takes too long,

//it will cause the watchdog to reset and the USB to reconnect

/*illustrate

buffer The buffer will be destroyed after the callback is completed, so you need to copy the data of the buffer yourself

buffer_length The length of the current buffer

failed Whether the acquisition failed

success Whether the collection is complete

need_total_sample A total of data needs to be collected

total_sample Data that has been collected

memoryuse How many buffers are currently used by the dynamic library (the more it is, the more data it means the dynamic library has piled up)*/

```

//Process data in the buffer according to your needs.....

//The last channel callback is complete and ready to exit
if(success)
{
    capture_ok_mask |= (0x01<<channel_index);
    if(capture_ok_mask==capture_channel_mask)
        capture_ok = true; //flag complete, and the main function exits
    }
}
}

```

10. Stream Mode API

10.1. Working mode

int SetOscCaptureMode(unsigned int dev_id, int is_stream_mode);

Description This routines set the osc capture mode.

Input: dev_id the dev id
 is_stream_mode enable the stream mode or not
 Output Return value 1 Success
 0 Failed

10.2. Collection range

The preamplifier of the device is equipped with a programmable gain amplifier, when the collected signal is smaller than the AD range, the gain amplifier can amplify the signal, and make more use of the number of AD bits to improve the quality of the acquired signal. The DLL will automatically adjust the gain amplifier of the preamp according to the set acquisition range.

int GetStreamRangeMinmV(unsigned int dev_id);

int GetStreamRangeMaxmV(unsigned int dev_id);

Description This routines set the range of input signal.

Output Return value
 minmv the minimum voltage of the input signal (mV)
 maxmv the maximum voltage of the input signal (mV)

int SetStreamChannelRangemV(unsigned int dev_id, int channel, int minmv, int maxmv);

Description This routines set the range of input signal.

Input: channel the set channel
 0 channel 1
 1 channel 2
 minmv the minimum voltage of the input signal (mV)
 maxmv the maximum voltage of the input signal (mV)
 Output Return value 1 Success
 0 Failed

Note: When the maximum acquisition range is probe X1, the maximum voltage that the oscilloscope can collect. For example, MSO20 is [-12000mV, 12000mV].

Note: In order to achieve a better waveform effect, you must set the acquisition range according to the amplitude of the waveform being measured. If necessary, the collection range can be dynamically changed.

10.3. Sampling rate

int GetStreamSupportSampleRateNum(unsigned int dev_id);

Description This routines get the number of samples that the equipment support.

Input: -

Output Return value the sample number

int GetStreamSupportSampleRates(unsigned int dev_id, unsigned int* sample, int maxnum);

Description This routines get support samples of equipment.

Input: sample the array store the support samples of the equipment

maxnum the length of the array

Output Return value the sample number of array stored

10.4. Capture

int StreamCapture(unsigned int dev_id, unsigned long long int length_kb, unsigned short capture_channel, unsigned int sample_rate);

Description This routines set the capture length and start capture.

Input: length capture length(KB)

capture_channel: //ch1=0x0001 ch2=0x0020 ch3=0x0040 ch4=0x0080

logic=0x0100

ch1+ch2 0x03

ch1+ch2+ch3 0x07

ch1+ch2+ch3+ch4 0x0F

Output Return 1 success

-1 sample_rate error

-2 device id error

void StreamStopCapture(unsigned int dev_id);

Description This routines stop the capture

Input:

Output Return

10.5. Capture completion notification

When data acquisition is complete, the rollback function will call back the notification and provide the corresponding acquisition buffer and acquisition status.

Void SetStreamDataReadyCallBack(void* ppara, StreamDataReadyCallBack datacallback);

Description This routines sets the callback function of capture complete.

Input: ppara the parameter of the callback function

datacallback a pointer to a function with the

StreamDataReadyCallBack prototype:

Output -

```
void CALLBACK StreamDataReadyCallBack(void* ppara, unsigned int dev_id,  
unsigned char channel_index, double* buffer, unsigned int buffer_length, unsigned int  
failed, unsigned int success, unsigned long long int need_total_sample, unsigned long  
long int total_sample, unsigned long long int memoryuse);
```

illustrate

buffer The buffer will be destroyed after the callback is completed, so you need to copy the data of the buffer yourself

buffer_length The length of the current buffer

failed Whether the acquisition failed

success Whether the collection is complete

need_total_sample A total of data needs to be collected

total_sample Data that has been collected

memoryuse How many buffers are currently used by the dynamic library (the more it is, the more data it means the dynamic library has piled up)